

Finals Activity 1

Marlon Alen I Flores
BSCS - 3A

CS311
2026-09-20

Contents

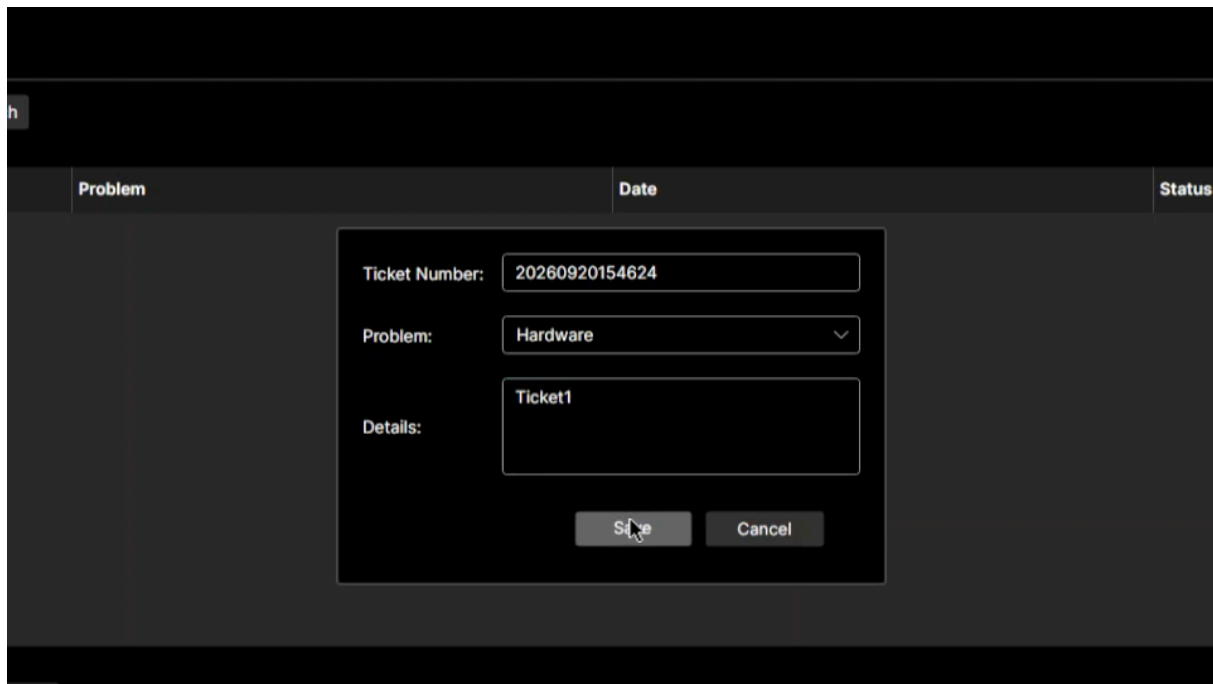
Screen Recording	1
Scenarios	1
Code	6

Screen Recording

<https://drive.google.com/file/d/1-kCQODRja99qOFla8jTgLyKT6rw4HjRL/view?usp=sharing>

Scenarios

Create 4 new tickets (2 hardware, 1 software, and 1 connection).



The screenshot shows a dark-themed user interface for creating a ticket. A modal form is centered on the screen, overlaid on a table with columns 'Problem', 'Date', and 'Status'. The form contains the following fields:

- Ticket Number:** A text input field containing the value '20260920154624'.
- Problem:** A dropdown menu with 'Hardware' selected.
- Details:** A text area containing the text 'Ticket1'.

At the bottom of the form are two buttons: 'Save' and 'Cancel'. A mouse cursor is hovering over the 'Save' button.

h

Problem	Date	Status
---------	------	--------

Ticket Number:

20260920154624

Problem:

Hardware

▼

Details:

Ticket1

Save

Cancel

h

Problem	Date	Status
---------	------	--------

Ticket Number:

20260920154648

Problem:

Software

▼

Details:

Ticket 3

Save

Cancel

h

Problem	Date	Status
---------	------	--------

Ticket Number: 20260920154700

Problem: Connection

Details: Ticket 4

SaveCancel

Highlights the tickets you have created on instruction 1.

Search: Search

Ticket Number	Problem	Date	Status
20260920154624	Hardware	9/20/2026 3:46:35 PM	Pending
20260920154638	Hardware	9/20/2026 3:46:47 PM	Pending
20260920154648	Software	9/20/2026 3:46:57 PM	Pending
20260920154700	Connection	9/20/2026 3:47:08 PM	Pending

AddUpdateDeleteRefresh

Search all tickets with hardware problem.

Search: Search

Ticket Number	Problem	Date	Status
20260920154638	Hardware	9/20/2026 3:46:47 PM	Pending
20260920160306	Hardware	9/20/2026 4:03:10 PM	Pending

Update the first ticket you have created into connection.

Search:

Ticket Number	Problem	Date	Status
20260920154624	Hardware		Pending
20260920154638	Hardware		Pending
20260920154648	Software		Pending
20260920154700	Connection		Pending

Ticket Number:

Problem:

Details:

Search:

Ticket Number	Problem	Date	Status
20260920154624	Hardware		Pending
20260920154638	Hardware		Pending
20260920154648	Software		Pending
20260920154700	Connection		Pending

Ticket Number:

Problem:

Details:

Search:

Ticket Number	Problem	Date	Status
20260920154624	Hardware		Pending
20260920154638	Hardware		Pending
20260920154648	Software		Pending
20260920154700	Connection		Pending

Ticket Number:

Problem:

Details:

Search:

Ticket Number	Problem	Date	Status
20260920154624	Connection	9/20/2026 3:46:35 PM	Pending
20260920154638	Hardware	9/20/2026 3:46:47 PM	Pending
20260920154648	Software	9/20/2026 3:46:57 PM	Pending
20260920154700	Connection	9/20/2026 3:47:08 PM	Pending

Search the first ticket you have created.

Search:

Ticket Number	Problem	Date	Status
20260920154624	Connection	9/20/2026 3:46:35 PM	Pending

Delete the first and last ticket you have created.

Search:

Ticket Number	Problem	Date	Status
20260920154624	Connection	9/20/2026 3:46:35 PM	Pending
20260920154638	Hardware	9/20/2026 3:46:47 PM	Pending
20260920154648	Software		Pending
20260920154700	Connection		Pending

Are you sure you want to delete this ticket?

Search:

Ticket Number	Problem	Date	Status
20260920154638	Hardware	9/20/2026 3:46:47 PM	Pending
20260920154648	Software	9/20/2026 3:46:57 PM	Pending
20260920154700	Connection		Pending

Are you sure you want to delete this ticket?

Show the remaining tickets.

Search:

Ticket Number	Problem	Date	Status
20260920154638	Hardware	9/20/2026 3:46:47 PM	Pending
20260920154648	Software	9/20/2026 3:46:57 PM	Pending

Show the logs table.

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Extra options

datelog	timelog	action	module	performedby	performedto
20/09/2026	3:46 PM	Created Ticket	Ticket Management	admin	20260920154624
20/09/2026	3:46 PM	Created Ticket	Ticket Management	admin	20260920154638
20/09/2026	3:46 PM	Created Ticket	Ticket Management	admin	20260920154648
20/09/2026	3:47 PM	Created Ticket	Ticket Management	admin	20260920154700
20/09/2026	3:47 PM	Updated Ticket	Ticket Management	admin	20260920154624
20/09/2026	3:47 PM	Delete Ticket	Ticket Management	admin	20260920154624
20/09/2026	3:47 PM	Delete Ticket	Ticket Management	admin	20260920154700

☐ Show all | Number of rows: 25 ▾ | Filter rows:

Code

Full code is available at <https://github.com/Alen-Flores/CS311-2026-FLORES/tree/master/Finals/CS311FinalsActivity1-Flores>

Models

./Models/Ticket.cs

```
1  using System;
2  using System.Data;
3
4  namespace CS311_CS3A_2026_Flores.Models;
5
6  public enum ProblemType
7  {
8      Hardware, Software, Connection
9  };
10
11 public enum StatusType
12 {
13     Pending
14 }
15
16
17 public record Ticket(
18     string TicketNumber,
19     ProblemType Problem,
20     string Details,
21     StatusType Status,
```

```

22     string CreatedBy,
23     DateTime DateCreated,
24     string? AssignedTo,
25     DateTime? DateAssigned,
26     DateTime? DateCompleted,
27     string? ApprovedBy,
28     DateTime? DateApproved
29 )
30 {
31
32     public static ProblemType[] ProblemTypes { get; }
33     = Enum.GetValues<ProblemType>();
34     public static Ticket FromDataRow(DataRow row)
35     {
36         return new Ticket(
37             row.Field<string>("TicketNumber")!,
38             Enum.Parse<ProblemType>(row.Field<string>("Problem")!),
39             row.Field<string>("Details")!,
40             Enum.Parse<StatusType>(row.Field<string>("Status")!),
41             row.Field<string>("CreatedBy")!,
42             row.Field<DateTime>("DateCreated")!,
43             row.Field<string?>("AssignedTo")!,
44             row.Field<DateTime?>("DateAssigned")!,
45             row.Field<DateTime?>("DateCompleted")!,
46             row.Field<string?>("ApprovedBy")!,
47             row.Field<DateTime?>("DateApproved")!
48         );
49     }
50 }

```

Services

./Services/TicketService.cs

```

1     using System;
2     using System.Collections.Generic;
3     using System.Data;
4     using System.Linq;
5     using CS311_CS3A_2026_Flores.Models;
6
7     namespace CS311_CS3A_2026_Flores.Services;
8     public interface ITicketService
9     {
10         List<Ticket> GetTickets();
11         List<Ticket> QueryTickets(string query, string username);
12         Ticket? GetTicket(int TicketNumber);
13         List<Ticket> GetTicketsByUser(string username);
14         void AddTicket(Ticket ticket);

```

```

15     void UpdateTicket(Ticket ticket);
16     void DeleteTicket(string TicketNumber);
17 }
18
19 public class TicketService : ITicketService
20 {
21
22     private DatabaseService dbService;
23     public TicketService(DatabaseService dbService)
24     {
25         this.dbService = dbService;
26     }
27
28     public Ticket? GetTicket(int TicketNumber)
29     {
30         DataTable dt = dbService.GetData($"
31         SELECT * FROM tbltickets
32         WHERE TicketNumber = '{TicketNumber}'
33         ");
34
35         DataRow? row = dt.AsEnumerable().FirstOrDefault();
36         if (row is null) return null;
37         return Ticket.FromDataRow(row);
38     }
39
40     public List<Ticket> GetTickets()
41     {
42         DataTable dt = dbService.GetData($"
43         SELECT * FROM tbltickets
44         ORDER BY DateApproved DESC
45         ");
46         return dt.AsEnumerable().Select(Ticket.FromDataRow).ToList();
47     }
48
49     public List<Ticket> GetTicketsByUser(string username)
50     {
51         DataTable dt = dbService.GetData($"
52         SELECT * FROM tbltickets
53         WHERE CreatedBy = '{username}'
54         ORDER BY DateApproved DESC
55         ");
56         return dt.AsEnumerable().Select(Ticket.FromDataRow).ToList();
57     }
58
59     public List<Ticket> QueryTickets(string query, string username)
60     {
61         DataTable dt = dbService.GetData($"

```

```

62     SELECT * FROM tbltickets
63     WHERE TicketNumber LIKE '%{query}%'
64     OR   Problem LIKE '%{query}%'
65     OR   Status LIKE '%{query}%'
66     AND CreatedBy = '{username}'
67     ORDER BY DateApproved DESC
68     """);
69     return dt.AsEnumerable().Select(Ticket.FromDataRow).ToList();
70 }
71
72 private string DatetimeOrNull(DateTime? dateTime)
73 {
74     if (dateTime.HasValue)
75     {
76         return $"\"{dateTime:yyyy-MM-dd HH:mm:ss}\"";
77     } else
78     {
79         return "NULL";
80     }
81 }
82
83 private string ToStringOrNull(object? ob)
84 {
85     if (ob is null)
86     {
87         return "NULL";
88     }
89     return "\"" + ob.ToString() + "\"";
90 }
91
92 public void AddTicket(Ticket ticket)
93 {
94     dbService.executeSQL($$$$
95     INSERT INTO tbltickets (
96         TicketNumber, Problem, Details,
97         Status, CreatedBy, DateCreated,
98         AssignedTo, DateAssigned, DateCompleted,
99         ApprovedBy, DateApproved
100     )
101     VALUES (
102         "{ticket.TicketNumber}",
103         "{ticket.Problem}",
104         "{ticket.Details}",
105         "{ticket.Status}",
106         "{ticket.CreatedBy}",
107         {DatetimeOrNull(ticket.DateCreated)},
108         {ToStringOrNull(ticket.AssignedTo)},

```

```

109         {DatetimeOrNull(ticket.DateAssigned)},
110         {DatetimeOrNull(ticket.DateCompleted)},
111         {ToStringOrNull(ticket.ApprovedBy)},
112         {DatetimeOrNull(ticket.DateApproved)}
113     )
114     """);
115 }
116
117 public void UpdateTicket(Ticket ticket)
118 {
119     dbService.executeSQL($""
120     UPDATE tbltickets
121     SET Problem = '{ticket.Problem}',
122     Details = '{ticket.Details}'
123     WHERE TicketNumber = '{ticket.TicketNumber}'
124     """);
125 }
126
127 public void DeleteTicket(string TicketNumber)
128 {
129     dbService.executeSQL($""
130     DELETE FROM tbltickets
131     WHERE TicketNumber = '{TicketNumber}'
132     """);
133 }
134 }

```

ViewModel

./ViewModels/Tickets/TicketsViewModel.cs

```

1  using System;
2  using System.Collections.Generic;
3  using System.Collections.ObjectModel;
4  using System.Threading.Tasks;
5  using Avalonia.Controls;
6  using CommunityToolkit.Mvvm.ComponentModel;
7  using CommunityToolkit.Mvvm.Input;
8  using CS311_CS3A_2026_Flores.Models;
9  using CS311_CS3A_2026_Flores.Services;
10 using CS311_CS3A_2026_Flores.Views;
11
12 namespace CS311_CS3A_2026_Flores.ViewModels;
13
14 public partial class TicketsViewModel : ObservableObject
15 {
16
17     [ObservableProperty]

```

```

18     object? _currentControl;
19
20     [ObservableProperty]
21     private List<Ticket> _tickets;
22
23     [ObservableProperty]
24     private int _selectedIndex;
25
26     private readonly ITicketService _ticketService;
27     private readonly IAuthService _authService;
28     private readonly ILoggingService _loggingService;
29
30     public TicketsViewModel(
31         ITicketService ticketService,
32         IAuthService authService,
33         ILoggingService loggingService
34     )
35     {
36         _ticketService = ticketService;
37         _authService = authService;
38         _loggingService = loggingService;
39         _tickets = ticketService.GetTicketsByUser(
40             authService.GetUser().GetAwaiter().GetResult().Username
41         );
42     }
43
44     private bool CurrentIsNull()
45     {
46         return CurrentControl is null;
47     }
48
49     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
50     private void Add()
51     {
52         var addTicketViewModel =
53             new AddTicketViewModel(_ticketService, _authService, _loggingService);
54         addTicketViewModel.exit += () => CurrentControl = null;
55         CurrentControl = addTicketViewModel;
56     }
57
58     [RelayCommand(CanExecute = nameof(CurrentIsNull))]
59     private void Update()
60     {
61         var target = Tickets[SelectedIndex];
62         var addTicketViewModel =
63             new UpdateTicketViewModel(_ticketService, _authService, _loggingService,
        target);

```

```

64     addTicketViewModel.exit += () => CurrentControl = null;
65     CurrentControl = addTicketViewModel;
66 }
67
68 [RelayCommand(CanExecute = nameof(CurrentIsNull))]
69 private async Task Delete()
70 {
71     var target = Tickets[SelectedIndex];
72     var choice = await Dialog.Show("Are you sure you want to delete this
73     ticket?",
74     Dialog.Buttons.YesNo);
75     switch (choice)
76     {
77         case Dialog.DialogResult.Yes:
78             _ticketService.DeleteTicket(target.TicketNumber);
79             _loggingService.LogAction(
80                 Log.WithCurrentTimeStamp("Delete Ticket", "Ticket Management",
81                 (await _authService.GetUser()).Username, target.TicketNumber
82                 ));
83             SelectedIndex = -1;
84             break;
85         default:
86             break;
87     }
88
89 [RelayCommand(CanExecute = nameof(CurrentIsNull))]
90 private async Task Refresh()
91 {
92     SelectedIndex = -1;
93     Tickets = _ticketService.GetTicketsByUser(
94         (await _authService.GetUser()).Username
95     );
96 }
97
98 [RelayCommand]
99 private async Task Search(string query)
100 {
101     Tickets = _ticketService.QueryTickets(
102         query,
103         (await _authService.GetUser()).Username
104     );
105 }
106 }

```

./ViewModels/Tickets/AddTicketViewModel.cs

```
1 using System;
```

C#

```

2  using System.ComponentModel.DataAnnotations;
3  using System.Threading.Tasks;
4  using CommunityToolkit.Mvvm.ComponentModel;
5  using CommunityToolkit.Mvvm.Input;
6  using CS311_CS3A_2026_Flores.Models;
7  using CS311_CS3A_2026_Flores.Services;
8  using CS311_CS3A_2026_Flores.Views;
9
10 namespace CS311_CS3A_2026_Flores.ViewModels;
11
12 public partial class AddTicketViewModel : ObservableValidator
13 {
14     [ObservableProperty]
15     private string _ticketNumber;
16
17     [ObservableProperty]
18     [Required]
19     private ProblemType? _problem;
20
21     [ObservableProperty]
22     private string? _details;
23
24     public event Action? exit;
25
26     private readonly ITicketService _ticketService;
27     private readonly IAuthService _authService;
28     private readonly ILoggingService _loggingService;
29
30     public AddTicketViewModel(
31         ITicketService ticketService,
32         IAuthService authService,
33         ILoggingService loggingService
34     )
35     {
36         _ticketNumber = DateTime.Now.ToString("yyyyMMddHHmmss");
37         _ticketService = ticketService;
38         _authService = authService;
39         _loggingService = loggingService;
40     }
41
42     [RelayCommand]
43     private async Task Save()
44     {
45         ValidateAllProperties();
46         if (HasErrors || Problem is null)
47         {
48             return;

```

```

49     }
50
51     _ticketService.AddTicket(new Ticket(
52         TicketNumber,
53         Problem.Value,
54         Details!,
55         StatusType.Pending,
56         (await _authService.GetUser()).Username,
57         DateTime.Now,
58         null, null, null, null, null
59     ));
60     _loggingService.LogAction(Log.WithCurrentTimeStamp(
61         "Created Ticket",
62         "Ticket Management",
63         (await _authService.GetUser()).Username,
64         TicketNumber
65     ));
66     await Dialog.Show("Created new ticket", Dialog.Buttons.Ok);
67     exit?.Invoke();
68 }
69
70 [RelayCommand]
71 private void Cancel()
72 {
73     exit?.Invoke();
74 }
75 }

```

./ViewModels/Tickets/UpdateTicketViewModel.cs

```

1  using System;
2  using System.ComponentModel.DataAnnotations;
3  using System.Threading.Tasks;
4  using CommunityToolkit.Mvvm.ComponentModel;
5  using CommunityToolkit.Mvvm.Input;
6  using CS311_CS3A_2026_Flores.Models;
7  using CS311_CS3A_2026_Flores.Services;
8  using CS311_CS3A_2026_Flores.Views;
9
10 namespace CS311_CS3A_2026_Flores.ViewModels;
11
12 public partial class UpdateTicketViewModel : ObservableValidator
13 {
14
15     public event Action? exit;
16
17     [ObservableProperty]

```

C#

```

18     private Ticket _target;
19
20     [ObservableProperty]
21     [Required]
22     private ProblemType? _problem;
23
24     [ObservableProperty]
25     private string? _details;
26
27     private readonly ITicketService _ticketService;
28     private readonly IAuthService _authService;
29     private readonly ILoggingService _loggingService;
30
31     public UpdateTicketViewModel(
32         ITicketService ticketService,
33         IAuthService authService,
34         ILoggingService loggingService,
35         Ticket target
36     )
37     {
38         _ticketService = ticketService;
39         _authService = authService;
40         _loggingService = loggingService;
41         _target = target;
42         _problem = target.Problem;
43         _details = target.Details;
44     }
45
46     [RelayCommand]
47     public async Task Save()
48     {
49         ValidateAllProperties();
50         if (HasErrors || Problem is null)
51         {
52             return;
53         }
54
55         _ticketService.UpdateTicket(Target with {
56             Problem = Problem.Value,
57             Details = Details!
58         });
59         _loggingService.LogAction(Log.WithCurrentTimeStamp(
60             "Updated Ticket",
61             "Ticket Management",
62             (await _authService.GetUser()).Username,
63             Target.TicketNumber
64         ));

```

```
65     await Dialog.Show("Updated ticket", Dialog.Buttons.Ok);
66     exit?.Invoke();
67 }
68
69 [RelayCommand]
70 public void Cancel()
71 {
72     exit?.Invoke();
73 }
74 }
```